

Code-Layout, Readability and Reusability

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation throughout server.py and all ml/ scripts. PEP-8 compliant.
2	Proper File Structure	Files and folders are logically organised	Yes	Root: server.py, requirements.txt, README.md. Subfolders: data/ (CSV, JSON), models/ (joblib, PNG), ml/ (train/predict scripts), web/ (HTML, CSS, JS, assets), scripts/ (setup helpers).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables like crop_model, disease_model, fertilizer_df, probability_matrix, weighted_score, remediation, nutrient_balance are all self-descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	Functions: get_fertilizer_recommendation(), evaluate_crop_suitability(), get_weather_data(), build_analytics_summary(), predict_image_disease_with_keras(), load_dataset(), score_metric(), safe_resolve().
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings on key functions (e.g., safe_resolve: 'Resolve a URL path to a file path, preventing directory traversal'). Inline comments explain ML steps and API logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	server.py: each feature is a separate function. ml/ folder: separate training scripts per model (train_model.py, train_disease_model.py, train_image_model.py). Consistent handler pattern in OptiCropHandler.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared utilities (load_dataset(), get_fertilizer_recommendation(), score_metric()) are defined once and reused across multiple API endpoints.

8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/except blocks on all model inference calls. HTTP 400 for invalid JSON. HTTP 503 if model file missing. HTTP 502 if weather API fails. Graceful fallback crop response if model unavailable.
---	----------------	--	-----	---

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	load_dataset()	Python	Used by evaluate_crop_suitability(), build_analytics_summary(), and get_crop_records(). Loads from CSV or JSON fallback.	High
2	get_fertilizer_recommendation()	Python	Called by /api/recommend, /api/suitability, and /api/fertilizer — same function serves all three endpoints.	High
3	score_metric()	Python	Reused for all 7 environmental metrics inside evaluate_crop_suitability(). Calculates normalised fit score for any parameter.	High
4	safe_resolve()	Python	Called by do_GET() and do_HEAD() to prevent directory traversal for every static file request.	High
5	crop_model.joblib	Joblib bundle	Loaded once at startup; reused across all /api/recommend requests without reloading. Contains RF, DT, SVC models + encoders.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Single-file server with clearly separated handler methods. Clean folder hierarchy (web/, ml/, models/, data/, scripts/).
Readability	5	All functions descriptively named. Docstrings on utility functions. Inline comments on complex ML and API logic.
Reusability	5	Core functions (load_dataset, get_fertilizer_recommendation, score_metric, safe_resolve) reused across multiple endpoints without duplication.
Documentation / Comments	4	README.md covers setup, run instructions, and dataset download. Docstrings on key functions. Inline comments explain non-obvious logic. Improvement: add per-module docstrings.
Overall Score	4.75	High-quality, well-structured, maintainable codebase suitable for extension and academic evaluation.